11.7 The QR Algorithm for Real Hessenberg Matrices

To complete the strategy for real, nonsymmetric matrices that was laid out in §11.6, we need to compute the eigenvalues and eigenvectors of a real Hessenberg matrix. Recall the following relations for the QR algorithm with shifts:

$$\mathbf{Q}_s \cdot (\mathbf{A}_s - k_s \mathbf{1}) = \mathbf{R}_s \tag{11.7.1}$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular, and

$$\begin{aligned} \mathbf{A}_{s+1} &= \mathbf{R}_s \cdot \mathbf{Q}_s^T + k_s \mathbf{1} \\ &= \mathbf{Q}_s \cdot \mathbf{A}_s \cdot \mathbf{Q}_s^T \end{aligned} \tag{11.7.2}$$

The QR transformation preserves the upper Hessenberg form of the original matrix $\mathbf{A} \equiv \mathbf{A}_1$, and the workload on such a matrix is $O(n^2)$ per iteration as opposed to $O(n^3)$ on a general matrix. As $s \to \infty$, $\mathbf{A}_s$ converges to a form where the eigenvalues are either isolated on the diagonal or are eigenvalues of a 2×2 submatrix on the diagonal.

As we pointed out in §11.4, shifting is essential for rapid convergence. A key difference here is that a nonsymmetric real matrix can have complex eigenvalues. This means that good choices for the shifts k_s may be complex, apparently necessitating complex arithmetic.

Complex arithmetic can be avoided, however, by a clever trick. This trick, plus a detailed description of how the QR algorithm is used, is described in a Webnote [1].

The operation count for the QR algorithm for Hessenberg matrices is $\sim 10k^2$ per iteration, where k is the current size of the matrix. The typical average number of iterations per eigenvalue is about two, so the total operation count for all the eigenvalues is $\sim 10n^3$. The total operation count for both eigenvalues and eigenvectors is $\sim 25n^3$.

The routines `hqr` for the eigenvalues only, and `hqr2`, which computes both eigenvalues and eigenvectors, are given in full in a Webnote [2], along with a few `Unsymmeig` utility routines not already listed. The implementations are based algorithmically on the above description, in turn following the implementations in [3,4].

CITED REFERENCES AND FURTHER READING:

Numerical Recipes Software 2007, "Description of the QR Algorithm for Hessenberg Matrices," *Numerical Recipes Webnote No. 16*, at `http://www.nr.com/webnotes?16` [1]

Numerical Recipes Software 2007, "Implementations in Unsymmeig," *Numerical Recipes Webnote No. 17*, at `http://www.nr.com/webnotes?17` [2]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[3]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §7.5.

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[4]

11.8 Improving Eigenvalues and/or Finding Eigenvectors by Inverse Iteration

The basic idea behind inverse iteration is quite simple. Let $\mathbf{y}$ be the solution of the linear system

$$(\mathbf{A} - \tau\mathbf{1}) \cdot \mathbf{y} = \mathbf{b} \tag{11.8.1}$$

where $\mathbf{b}$ is a random vector and τ is close to some eigenvalue λ of $\mathbf{A}$. Then the solution $\mathbf{y}$ will be close to the eigenvector corresponding to λ. The procedure can be iterated: Replace $\mathbf{b}$ by $\mathbf{y}$ and solve for a new $\mathbf{y}$, which will be even closer to the true eigenvector.

We can see why this works by expanding both $\mathbf{y}$ and $\mathbf{b}$ as linear combinations of the eigenvectors $\mathbf{x}_j$ of $\mathbf{A}$:

$$\mathbf{y} = \sum_j \alpha_j \mathbf{x}_j \qquad \mathbf{b} = \sum_j \beta_j \mathbf{x}_j \tag{11.8.2}$$

Then (11.8.1) gives

$$\sum_j \alpha_j (\lambda_j - \tau)\mathbf{x}_j = \sum_j \beta_j \mathbf{x}_j \tag{11.8.3}$$

so that

$$\alpha_j = \frac{\beta_j}{\lambda_j - \tau} \tag{11.8.4}$$

and

$$\mathbf{y} = \sum_j \frac{\beta_j \mathbf{x}_j}{\lambda_j - \tau} \tag{11.8.5}$$

If τ is close to λ_n, say, then provided β_n is not accidentally too small, $\mathbf{y}$ will be approximately $\mathbf{x}_n$, up to a normalization. Moreover, each iteration of this procedure gives another power of $\lambda_j - \tau$ in the denominator of (11.8.5). Thus the convergence is rapid for well-separated eigenvalues.

Suppose at the kth stage of iteration we are solving the equation

$$(\mathbf{A} - \tau_k\mathbf{1}) \cdot \mathbf{y} = \mathbf{b}_k \tag{11.8.6}$$

where $\mathbf{b}_k$ and τ_k are our current guesses for some eigenvector and eigenvalue of interest (let's say $\mathbf{x}_n$ and λ_n). Normalize $\mathbf{b}_k$ so that $\mathbf{b}_k \cdot \mathbf{b}_k = 1$. The exact eigenvector and eigenvalue satisfy

$$\mathbf{A} \cdot \mathbf{x}_n = \lambda_n \mathbf{x}_n \tag{11.8.7}$$

so

$$(\mathbf{A} - \tau_k\mathbf{1}) \cdot \mathbf{x}_n = (\lambda_n - \tau_k)\mathbf{x}_n \tag{11.8.8}$$

Since $\mathbf{y}$ of (11.8.6) is an improved approximation to $\mathbf{x}_n$, we normalize it and set

$$\mathbf{b}_{k+1} = \frac{\mathbf{y}}{|\mathbf{y}|} \tag{11.8.9}$$

We get an improved estimate of the eigenvalue by substituting our improved guess $\mathbf{y}$ for $\mathbf{x}_n$ in (11.8.8). By (11.8.6), the left-hand side is $\mathbf{b}_k$, so, calling λ_n our new value τ_{k+1}, we find

$$\tau_{k+1} = \tau_k + \frac{1}{\mathbf{b}_k \cdot \mathbf{y}} \tag{11.8.10}$$

While the above formulas look simple enough, in practice the implementation can be quite tricky. The first question to be resolved is *when* to use inverse iteration. Most of the computational load occurs in solving the linear system (11.8.6). Thus a possible strategy is first to reduce the matrix $\mathbf{A}$ to a special form that allows easy solution of (11.8.6). Tridiagonal form for symmetric matrices or Hessenberg for nonsymmetric are the obvious choices. The tridiagonal form can be solved in $O(N)$ operations, the Hessenberg form in $O(N^2)$ operations. If you then apply inverse iteration to generate all the eigenvectors, this gives an $O(N^2)$ method for tridiagonal matrices. The problem is that closely spaced eigenvalues lead to eigenvectors that are not properly orthogonal to one another. Using a procedure like Gram-Schmidt to orthogonalize the vectors is $O(n^3)$, and not entirely satisfactory anyway. Accordingly, inverse iteration is generally used when one already has good eigenvalues and wants only a few selected eigenvectors.

You can write a simple inverse iteration routine yourself using LU decomposition to solve (11.8.6). You can decide whether to use the general LU algorithm we gave in Chapter 2 or whether to take advantage of tridiagonal or Hessenberg form. Note that, since the linear system (11.8.6) is nearly singular, you must be careful to use a version of LU decomposition like that in §2.3 which replaces a zero pivot with a very small number.

We have chosen not to give a general inverse iteration routine in this book, because it is quite cumbersome to take account of all the cases that can arise. Routines are given, for example, in [1-3]. If you use these, or write your own routine, you may appreciate the following pointers.

One starts by supplying an initial value τ_0 for the eigenvalue λ_n of interest. Choose a random normalized vector $\mathbf{b}_0$ as the initial guess for the eigenvector $\mathbf{x}_n$, and solve (11.8.6). The new vector $\mathbf{y}$ is bigger than $\mathbf{b}_0$ by a "growth factor" $|\mathbf{y}|$, which ideally should be large. Equivalently, the change in the eigenvalue, which by (11.8.10) is essentially $1/|\mathbf{y}|$, should be small. The following cases can arise:

- If the growth factor is too small initially, then we assume we have made a "bad" choice of random vector. This can happen not just because of a small β_n in (11.8.5), but also in the case of a defective matrix, when (11.8.5) does not even apply (see, e.g., [1] or [4] for details). We go back to the beginning and choose a new initial vector.
- The change $|\mathbf{b}_1 - \mathbf{b}_0|$ might be less than some tolerance ϵ. We can use this as a criterion for stopping, iterating until it is satisfied, with a maximum of 5–10 iterations, say.
- After a few iterations, if $|\mathbf{b}_{k+1} - \mathbf{b}_k|$ is not decreasing rapidly enough, we can try updating the eigenvalue according to (11.8.10). If $\tau_{k+1} = \tau_k$ to machine accuracy, we are not going to improve the eigenvector much more and can quit. Otherwise start another cycle of iterations with the new eigenvalue.

The reason we do not update the eigenvalue at every step is that when we solve the linear system (11.8.6) by LU decomposition, we can save the decomposition if

τ_k is fixed (assuming we are working with the full matrix). We only need to do the backsubstitution step each time we update $\mathbf{b}_k$. The number of iterations we decide to do with a fixed τ_k is a trade-off between the quadratic convergence but $O(N^3)$ workload for updating τ_k at each step and the linear convergence but $O(N^2)$ load for keeping τ_k fixed. If you have determined the eigenvalue by one of the routines given earlier in the chapter, it is probably correct to machine accuracy anyway, and you can omit updating it.

There are two different pathologies that can arise during inverse iteration. The first is multiple or closely spaced roots. This is more often a problem with symmetric matrices. Inverse iteration will find only one eigenvector for a given initial guess τ_0. A good strategy is to perturb the last few significant digits in τ_0 and then repeat the iteration. Usually this provides an independent eigenvector. Special steps generally have to be taken to ensure orthogonality of the linearly independent eigenvectors, whereas the Jacobi and QL algorithms automatically yield orthogonal eigenvectors even in the case of multiple eigenvalues.

The second problem, peculiar to nonsymmetric matrices, is the defective case. Unless one makes a "good" initial guess, the growth factor is small. Moreover, iteration does not improve matters. In this case, the remedy is to choose random initial vectors, solve (11.8.6) once, and quit as soon as *any* vector gives an acceptably large growth factor. Typically only a few trials are necessary.

One further complication in the nonsymmetric case is that a real matrix can have complex-conjugate pairs of eigenvalues. You will then have to use complex arithmetic to solve (11.8.6) for the complex eigenvectors. For any moderate-sized (or larger) nonsymmetric matrix, our recommendation is to avoid inverse iteration in favor of a QR method like `Unsymmeig`.

A good discussion of these and other problems with inverse iteration is given in [5]. As discussed in §11.4.4, for symmetric tridiagonal matrices, the MRRR algorithm is a sophisticated version of inverse iteration that avoids all these problems.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America).

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), p. 418.[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, `http://www.netlib.org/lapack`.[3]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.6.3.[4]

Dhillon, I.S. 1998, "Current Inverse Iteration Software Can Fail," *BIT Numerical Mathematics*, vol. 38, pp. 685–704.[5]

Fast Fourier Transform

CHAPTER 12

12.0 Introduction

A very large class of important computational problems falls under the general rubric of *Fourier transform methods* or *spectral methods*. For some of these problems, the Fourier transform is simply an efficient computational tool for accomplishing certain common manipulations of data. In other cases, we have problems for which the Fourier transform (or the related *power spectrum*) is itself of intrinsic interest. These two kinds of problems share a common methodology.

Historically, Fourier and spectral methods have been considered a part of "signal processing," rather than "numerical analysis" proper. There is really no justification for such a distinction. Fourier methods are commonplace in research and we will not treat them as specialized or arcane. However, we realize that many users have had relatively less experience with this field than with, say, differential equations or numerical integration. Therefore our summary of analytical results will be more complete. Numerical algorithms, per se, begin in §12.2. Various applications of Fourier transform methods are discussed in Chapter 13.

A physical process can be described either in the *time domain*, by the values of some quantity h as a function of time t, e.g., $h(t)$, or else in the *frequency domain*, where the process is specified by giving its amplitude H (generally a complex number indicating phase also) as a function of frequency f, that is, $H(f)$, with $-\infty < f < \infty$. For many purposes it is useful to think of $h(t)$ and $H(f)$ as being two different *representations* of the same function. One goes back and forth between these two representations by means of the *Fourier transform* equations,

$$\begin{aligned} H(f) &= \int_{-\infty}^{\infty} h(t) e^{2\pi i f t}\, dt \\ h(t) &= \int_{-\infty}^{\infty} H(f) e^{-2\pi i f t}\, df \end{aligned} \tag{12.0.1}$$

If t is measured in seconds, then f in equation (12.0.1) is in cycles per second, or Hertz (the unit of frequency). However, the equations work with other units, too. If h is a function of position x (in meters), H will be a function of inverse wavelength

(cycles per meter), and so on. If you are trained as a physicist or mathematician, you are probably more used to using *angular frequency* ω, which is given in *radians* per second. The relation between ω and f, $H(\omega)$ and $H(f)$, is

$$\omega \equiv 2\pi f \qquad H(\omega) \equiv [H(f)]_{f=\omega/2\pi} \tag{12.0.2}$$

and equation (12.0.1) looks like this:

$$\begin{aligned} H(\omega) &= \int_{-\infty}^{\infty} h(t) e^{i\omega t}\, dt \\ h(t) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} H(\omega) e^{-i\omega t}\, d\omega \end{aligned} \tag{12.0.3}$$

We were raised on the ω-convention, but we changed! There are fewer factors of 2π to remember if you use the f-convention, especially when we get to discretely sampled data in §12.1.

From equation (12.0.1) it is evident at once that Fourier transformation is a *linear* operation. The transform of the sum of two functions is equal to the sum of the transforms. The transform of a constant times a function is that same constant times the transform of the function.

In the time domain, the function $h(t)$ may happen to have one or more special symmetries. It might be *purely real* or *purely imaginary* or it might be *even*, $h(t) = h(-t)$, or *odd*, $h(t) = -h(-t)$. In the frequency domain, these symmetries lead to relationships between $H(f)$ and $H(-f)$. The following table gives the correspondence between symmetries in the two domains:

If . . .	then . . .
$h(t)$ is real	$H(-f) = [H(f)]^*$
$h(t)$ is imaginary	$H(-f) = -[H(f)]^*$
$h(t)$ is even	$H(-f) = H(f)$ [i.e., $H(f)$ is even]
$h(t)$ is odd	$H(-f) = -H(f)$ [i.e., $H(f)$ is odd]
$h(t)$ is real and even	$H(f)$ is real and even
$h(t)$ is real and odd	$H(f)$ is imaginary and odd
$h(t)$ is imaginary and even	$H(f)$ is imaginary and even
$h(t)$ is imaginary and odd	$H(f)$ is real and odd

In subsequent sections we shall see how to use these symmetries to increase computational efficiency.

Here are some other elementary properties of the Fourier transform. (We'll use the "$\Longleftrightarrow$" symbol to indicate transform pairs.) If

$$h(t) \Longleftrightarrow H(f) \tag{12.0.4}$$

is such a pair, then other transform pairs are

$$h(at) \Longleftrightarrow \frac{1}{|a|} H\left(\frac{f}{a}\right) \qquad \text{time scaling} \tag{12.0.5}$$

$$\frac{1}{|b|} h\left(\frac{t}{b}\right) \Longleftrightarrow H(bf) \qquad \text{frequency scaling} \tag{12.0.6}$$

$$h(t - t_0) \Longleftrightarrow H(f)\, e^{2\pi i f t_0} \qquad \text{time shifting} \tag{12.0.7}$$

$$h(t)\, e^{-2\pi i f_0 t} \Longleftrightarrow H(f - f_0) \qquad \text{frequency shifting} \tag{12.0.8}$$

With two functions $h(t)$ and $g(t)$, and their corresponding Fourier transforms $H(f)$ and $G(f)$, we can form two combinations of special interest. The *convolution* of the two functions, denoted $g * h$, is defined by

$$g * h \equiv \int_{-\infty}^{\infty} g(\tau) h(t - \tau)\, d\tau \tag{12.0.9}$$

Note that $g * h$ is a function in the time domain and that $g * h = h * g$. It turns out that the function $g * h$ is one member of a simple transform pair,

$$g * h \Longleftrightarrow G(f) H(f) \qquad \text{convolution theorem} \tag{12.0.10}$$

In other words, the Fourier transform of the convolution is just the product of the individual Fourier transforms.

The *correlation* of two functions, denoted $\mathrm{Corr}(g, h)$, is defined by

$$\mathrm{Corr}(g, h) \equiv \int_{-\infty}^{\infty} g(\tau + t) h(\tau)\, d\tau \tag{12.0.11}$$

The correlation is a function of t, which is called the *lag*. It therefore lies in the time domain, and it turns out to be one member of the transform pair:

$$\mathrm{Corr}(g, h) \Longleftrightarrow G(f) H^*(f) \qquad \text{correlation theorem} \tag{12.0.12}$$

[More generally, the second member of the pair is $G(f)H(-f)$, but we are restricting ourselves to the usual case in which g and h are real functions, so we take the liberty of setting $H(-f) = H^*(f)$.] This result shows that multiplying the Fourier transform of one function by the complex conjugate of the Fourier transform of the other gives the Fourier transform of their correlation. The correlation of a function with itself is called its *autocorrelation*. In this case (12.0.12) becomes the transform pair

$$\mathrm{Corr}(g, g) \Longleftrightarrow |G(f)|^2 \qquad \text{Wiener-Khinchin theorem} \tag{12.0.13}$$

The *total power* in a signal is the same whether we compute it in the time domain or in the frequency domain. This result is known as *Parseval's theorem*:

$$\text{total power} \equiv \int_{-\infty}^{\infty} |h(t)|^2\, dt = \int_{-\infty}^{\infty} |H(f)|^2\, df \tag{12.0.14}$$

Frequently one wants to know "how much power" is contained in the frequency interval between f and $f + df$. In such circumstances, one does not usually distinguish between positive and negative f, but rather regards f as varying from 0 ("zero frequency" or D.C.) to $+\infty$. In such cases, one defines the *one-sided power spectral density (PSD)* of the function h as

$$P_h(f) \equiv |H(f)|^2 + |H(-f)|^2 \qquad 0 \le f < \infty \tag{12.0.15}$$